3. Visualising Data

Extending Exploratory Data Analysis with Figures

2026/04/04

Table of contents

1 Introduction **1**

2 Key Concepts **2**

3 Choosing the Right Plot **2**

4 Visualising Distributions **2**
4.1 Histograms . . . 2
4.2 Density Plots . . . 4

5 Comparing Groups **5**
5.1 Box Plots . . . 5
5.2 Violin Plots . . . 7
5.3 Bar Graphs . . . 9

6 Visualising Relationships **10**
6.1 Scatter Plots . . . 10
6.2 Line Graphs . . . 11

7 Extensions **12**
7.1 Heatmaps . . . 12
7.2 Hovmöller Diagrams . . . 13
7.3 Other Examples (To Develop) . . . 15

8 Conclusion **15**

IN THIS CHAPTER

- The diversity of graphs used to communicate statistical results
- How to select the right graph for any particular dataset
- Additional packages available to extend ggplot's functionality

TASKS TO COMPLETE IN THIS CHAPTER

- Self-Assessment Task 3-1 **(/30)**
- Self-Assessment instructions and full task overview

1 Introduction

Figures are not there to look pretty, although they can. In exploratory data analysis, figures are part of the analysis itself. A good figure lets you see shape, spread, clustering, outliers, and relationships that a table of summary statistics cannot show.

In the work below, I will extend the summaries in Chapter 2 by turning them into graphical supporitng evidence. The main task is to match the plot to the question. Some plots show the distribution of one variable, some compare groups, and others reveal relationships between variables.

The examples here will help you recognise which figure answers which descriptive question, rather than to exhaust every possible `ggplot2` option. For broader coverage and additional figure types, consult the following sources:

- ggplot2 by Hadley Wickham et al.
- ggplot2: Elegant Graphics for Data Analysis, v.3 by Hadley Wickham, et al.
- R Graphics Cookbook, 2nd edition by Winston Chang
- A Comprehensive Guide on ggplot2 in R by Devashree Madhugiri
- ggplot2 extensions
- The R Graph Gallery by Yan Holtz

2 Key Concepts

These decision steps guide the figure types you will encounter below:

- **Graphical EDA:** Plots are an essential part of the data exploration and assist in communicating our findings.
- **`ggplot2`:** A versatile R package for building statistical graphics.
- **Distribution Plots:** Use histograms and density plots to examine the shape of a single variable.
- **Group Comparison Plots:** Use box plots and violin plots to compare distributions across groups; use bar graphs mainly for counts or proportions.
- **Relationship Plots:** Use scatter plots and line graphs to show how variables change together.

3 Choosing the Right Plot

Choose the plot from the question you are asking. So, if you want to:

- know how a single variable is distributed, start with a **histogram** or **density plot**.
- compare groups, use a **box plot** or **violin plot**. Use a **bar graph** only when the quantity of interest is a count or proportion, or when you deliberately want to show a summary such as a mean with error bars.
- examine the relationship between two variables, use a **scatter plot**. If one axis is ordered in time or sequence, a **line graph** may be better.

Summary statistics remain important, but they are not enough on their own because two datasets can share the same mean, standard deviation, and correlation while having completely different structure.

DO IT NOW!

Retrieve the `random_not.csv` and `random_not_either.csv` data files I made for this exercise and calculate the mean and standard deviation of the variables in each file. Then make scatter plots of the two datasets side by side and compare them. The contrast is obvious. What do you see? What do you conclude?
The point of the exercise is that a figure can expose structure that summary statistics conceal.

4 Visualising Distributions

We begin with plots that describe the shape of a single variable. These plots answer questions about skewness, spread, outliers, and possible multi-modal distribution.

4.1 Histograms

Histograms display the distribution of a continuous variable by dividing the data into bins and counting how many observations fall within each bin. They should be the first plot to make when you want to inspect the shape of a distribution.

We have a choice of **absolute** frequency histograms, where the bar heights are counts, and **relative** frequency histograms, where the heights are proportions. Relative frequency is useful when comparing samples of different sizes. The **empirical cumulative distribution function** (ECDF) is another distribution plot that shows the cumulative proportion of observations below each value. The Old Faithful data illustrate these alternatives in Figure 1.

```
hist1 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_histogram(colour = "black", fill = "salmon", alpha = 0.6) +
  labs(title = "'Vanilla' Histogram",
       x = "Eruption duration (min)",
       y = "Count")

hist2 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_histogram(aes(y = ..density..),
                 position = "identity", binwidth = 1,
                 colour = "black", fill = "salmon", alpha = 0.6) +
  labs(title = "Relative Frequency",
       x = "Eruption duration (min)",
       y = "Relative\ncontribution")

hist3 <- ggplot(data = faithful, aes(x = waiting)) +
  geom_histogram(aes(y = 0.5 * ..density..),
                 position = "identity", binwidth = 0.5,
                 colour = "salmon", fill = "salmon", alpha = 0.6) +
  labs(title = "Relative Frequency",
       x = "Waiting time (min)",
       y = "Relative\ncontribution")

hist4 <- ggplot(data = faithful, aes(x = eruptions)) +
  stat_ecdf() +
  labs(title = "ECDF",
       x = "Eruption duration (min)",
       y = "Relative\ncontribution")

ggarrange(hist1, hist2, hist3, hist4, ncol = 2, nrow = 2, labels = "AUTO")
```

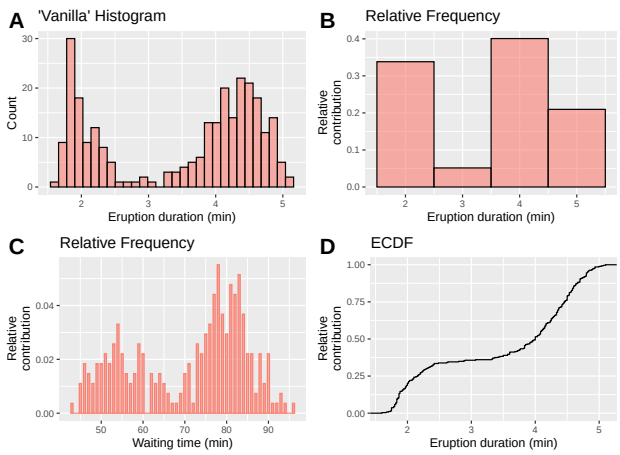

Figure 1. Example histograms for the Old Faithful data. A) A default frequency histogram. B) A relative frequency histogram with 1-minute bins. C) A relative frequency histogram with 0.5-minute bins. D) An ECDF.

`ggplot2` constructs histograms with `geom_histogram()`. We can also create bins manually with `cut()` if we need full control.

DO IT NOW!

Using with the `cut()` function, recreate Figure 1 A-C manually.

What if the same continuous variable is measured in several groups? The `iris` dataset gives a grouped example. These are size measurements for 50 flowers from each of three species of *Iris*. The grouped histograms in Figure 2 let us compare the distributions directly.

```
iris.long <- iris %>%
  gather(key = "variable", value = "size", -Species)

ggplot(data = iris.long, aes(x = size)) +
  geom_histogram(position = "dodge",
                 colour = NA, bins = 20,
                 aes(fill = Species)) +
  facet_wrap(~variable) +
  labs(x = "Size (cm)",
       y = "Count")
```

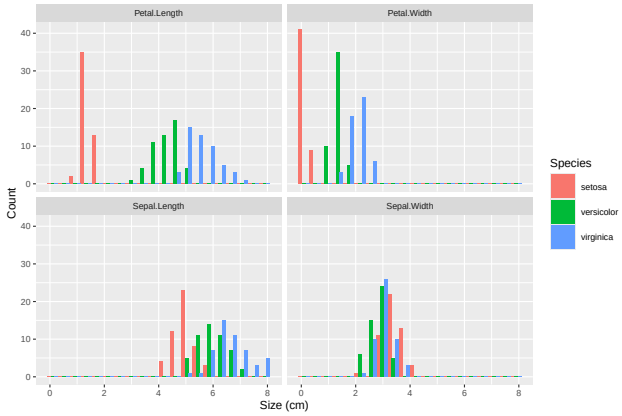

Figure 2. Grouped histograms for the four *Iris* variables.

4.2 Density Plots

Density plots provide a smoothed view of a distribution. They emphasise shape rather than bin counts, which makes them useful when you want to compare overall form without committing too strongly to a particular bin width.

Compared with a histogram, a density plot is smoother and often easier to compare across groups. The trade-off is that it does not show counts as directly. Histograms answer "how many observations fall in each interval?" Density plots answer "what is the overall shape of the distribution?"

Figure 3 shows several density-plot variations:

```
dens1 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_density(colour = "black", fill = "salmon", alpha = 0.6) +
  labs(x = "Eruption duration (min)",
       y = "Density")

dens2 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_density(colour = "black", fill = "salmon", alpha = 0.6) +
  geom_rug(colour = "red") +
  labs(x = "Eruption duration (min)",
       y = "Density")

dens3 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_histogram(aes(y = ..density..),
                 position = "identity", binwidth = 1,
```

```
                 colour = "black", fill = "turquoise", alpha = 0.6) +
  geom_density(colour = "black", fill = "salmon", alpha = 0.6) +
  labs(x = "Eruption duration (min)",
       y = "Density")

dens4 <- ggplot(data = faithful, aes(x = eruptions)) +
  geom_histogram(aes(y = ..count..),
                 binwidth = 0.2, colour = "black", fill = "turquoise", alpha = 0.6) +
  geom_density(aes(y = ..density.. * nrow(datasets::faithful) * 0.2),
               position = "identity",
               colour = "black", fill = "salmon", alpha = 0.6) +
  labs(x = "Eruption duration (min)",
       y = "Count")

ggarrange(dens1, dens2, dens3, dens4, ncol = 2, nrow = 2, labels = "AUTO")
```

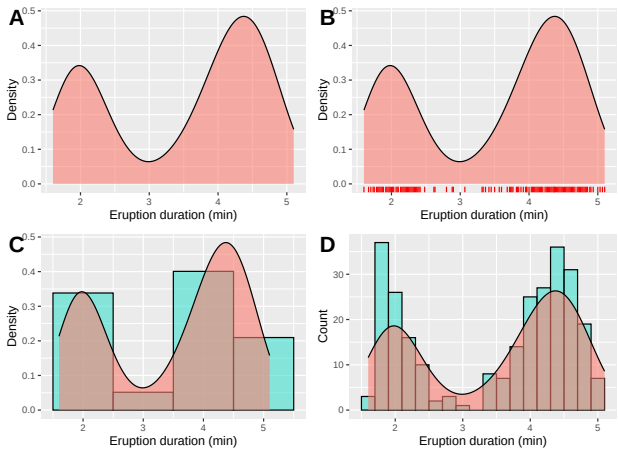

Figure 3. Density-plot variations based on the Old Faithful data. A) A basic density plot. B) A density plot with a rug. C) A histogram overlaid with a density curve. D) A count histogram with a rescaled density curve.

DO IT NOW!

Using the `faithful` dataset, produce two side-by-side plots: a histogram (choose a sensible bin width) and a density plot of `eruptions`. Look at the shapes. Do you see one peak or two? What does that tell you biologically? Now change the bin width in the histogram — does the apparent number of peaks change? Discuss with a partner whether the histogram or density plot is more revealing for this dataset and why.

5 Comparing Groups

We now compare distributions across categories. The main choice here is whether you want to show the full distribution within each group or only a summary. In most biological applications, the distribution is revealing.

5.1 Box Plots

Box plots provide a compact summary of the distribution within each group. They show the median, quartiles, whiskers, and potential outliers, which makes them useful for comparing several groups at once.

Box plots are often more informative than mean-only bar graphs because they show spread and asymmetry directly. A useful variation is to add jittered raw observations on top of the boxes.

I create a simple example using the `msleep` dataset (Figure 4). Additional examples are provided in Chapter 2.

```
msleep ▷
  ggplot(aes(x = vore, y = sleep_total)) +
  geom_boxplot(colour = "black", fill = "salmon", alpha = 0.4,
```

```
               outlier.colour = "red3", outlier.fill = "red",
               outlier.alpha = 1.0, outlier.size = 2.2) +
  geom_jitter(width = 0.10, fill = "blue", alpha = 0.5,
              col = "navy", shape = 21, size = 2.2) +
  labs(x = "'-vore'",
       y = "Sleep duration (hr)")
```

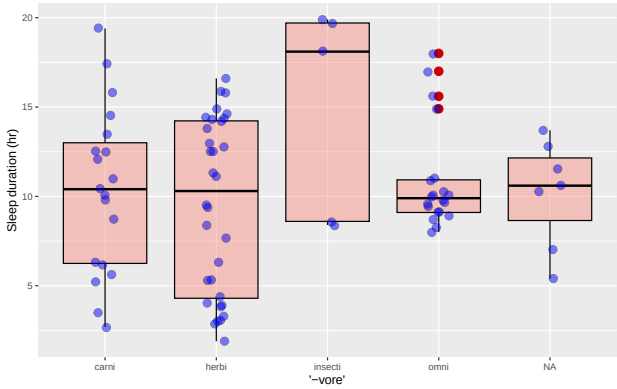

Figure 4. Box plot summarising the amount of sleep required by different 'vores'.

Box plots are sometimes called **box-and-whisker plots**. The `geom_boxplot()` documentation gives the clearest definition of the parts:

> The lower and upper hinges correspond to the first and third quartiles (the 25th and 75th percentiles).
>
> The upper whisker extends from the hinge to the largest value no further than 1.5 * IQR from the hinge (where IQR is the inter-quartile range, or distance between the first, third quartiles). The lower whisker extends from the hinge to the smallest value at most 1.5 * IQR of the hinge. Data beyond the end of the whiskers are called 'outlying' points and are plotted individually.
>
> In a notched box plot, the notches extend approximately $1.58 \times \text{IQR}/\sqrt{n}$. This gives a roughly 95% confidence interval for comparing medians.

Here is a second example, this time with notched box plots (Figure 5):

```
library(ggsci)

ggplot(data = iris.long, aes(x = Species, y = size)) +
  geom_boxplot(alpha = 0.4, notch = TRUE) +
  geom_jitter(width = 0.1, shape = 21, fill = NA,
              alpha = 0.4, aes(colour = as.factor(Species))) +
  facet_wrap(~variable, nrow = 2) +
  scale_color_npg() +
  labs(y = "Size (cm)") +
  guides(colour = FALSE) +
  theme(axis.text.x = element_text(face = "italic"))
```

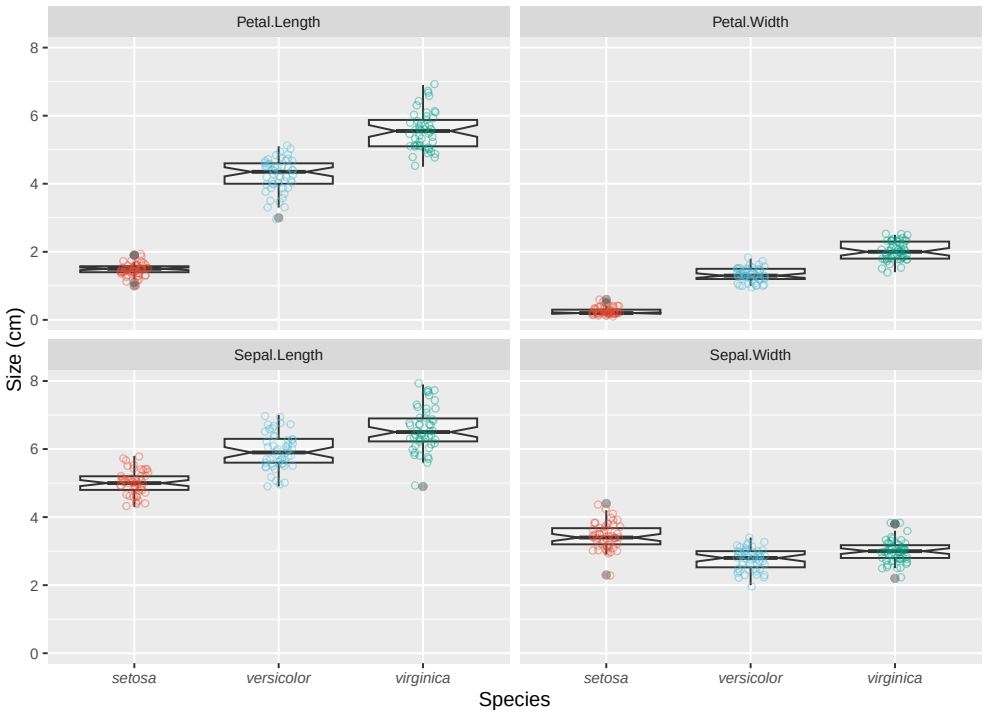

Figure 5. A panelled collection of box plots, one for each of the four variables, with a scatter plot to indicate the spread of the raw data points.

5.2 Violin Plots

Violin plots show the same grouped comparisons as box plots, but they display the full smoothed distribution within each group. That makes them useful when the shape of the data is interesting, especially if the distributions may be skewed or multimodal.

A box plot is better when you want a compact summary. A violin plot is better when you want to show the internal shape of the distribution. Here are the `iris` data shown with violin plots (Figure 6):

```
vio1 <- ggplot(data = iris, aes(x = Species, y = Sepal.Length, fill = Species)) +
  geom_violin() +
  labs(title = "Basic violin plot", y = "Sepal length (cm)") +
  theme(axis.text.x = element_text(face = "italic"),
        legend.position = "none")

vio2 <- ggplot(data = iris, aes(x = Species, y = Sepal.Length, fill = Species)) +
  geom_violin(show.legend = FALSE, draw_quantiles = c(0.25, 0.5, 0.75)) +
  labs(title = "With quartiles", y = "Sepal length (cm)") +
  theme(axis.text.x = element_text(face = "italic"),
        legend.position = "none")

vio3 <- ggplot(data = iris, aes(x = Species, y = Sepal.Length, colour = Species)) +
  geom_violin(fill = "grey70") +
  geom_boxplot(width = 0.1, colour = "grey30", fill = "white") +
  labs(title = "Nested box plots", y = "Sepal length (cm)") +
  theme(axis.text.x = element_text(face = "italic"),
        legend.position = "none")

vio4 <- ggplot(data = iris, aes(x = Species, y = Sepal.Length, colour = Species)) +
  geom_violin(fill = "grey70") +
  geom_boxplot(width = 0.1, colour = "black", fill = "white") +
  geom_jitter(shape = 1, width = 0.1, colour = "red", alpha = 0.7, fill = NA) +
  labs(title = "Boxes and jittered data", y = "Sepal length (cm)") +
  theme(axis.text.x = element_text(face = "italic"),
        legend.position = "none")
```

```
ggarrange(vio1, vio2, vio3, vio4, ncol = 2, nrow = 2, labels = "AUTO")
```

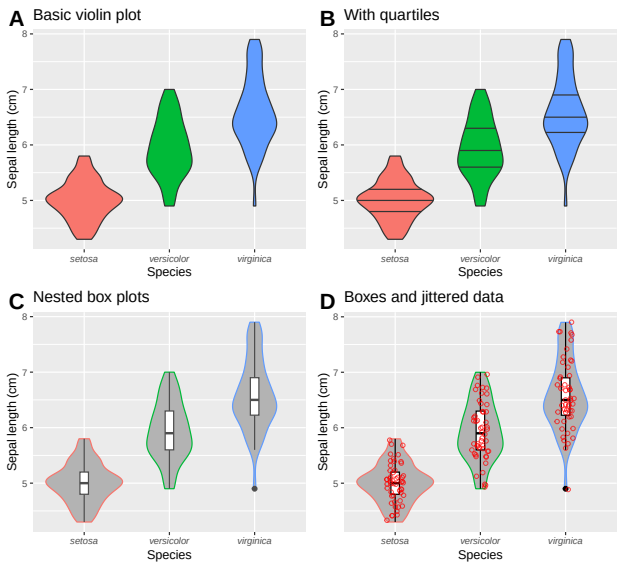

Figure 6. Variations of violin plots applied to the *Iris* data.

The **ggpubr** package also provides many convenience functions for publication-quality graphs, including violin plots.

DO IT NOW!

For each of the following scenarios, decide whether a box plot or a violin plot is more appropriate, and briefly justify your choice:

a. You have body-mass data for three penguin species (n ≈ 150 per species) and want to check whether the distributions are symmetric or skewed.
b. You are comparing gill-net catch-per-unit-effort across five coastal sites, but n = 8 per site.
c. A reviewer asks you to show the raw spread of pH measurements at two sites in a manuscript figure.

After deciding individually, compare your choices with a partner. Are there cases where both would be acceptable?

SELF-ASSESSMENT TASK 3-1

a. Using a tidy workflow, assemble a summary table of the **palmerpenguins** dataset that has a similar appearance as that produced by `psych::describe(penguins)`. **(/5)**
    - For bonus marks (which will not count anything), apply a beautiful, creative styling to the table using the **kable** package. Try and make it as publication ready as possible. Refer to a few journal articles to see how to professionally typeset tables.
b. Still using the **palmerpenguins** dataset, perform an exploratory data analysis to investigate the relationship between penguin species, their morphological traits (bill length and bill depth, flipper length, and body mass). Employ the tidyverse approaches learned earlier in the module to explore the data, account for the grouping structures present within the dataset. **(/10)**
c. Provide visualisations (use Figure 4 as inspiration) and summary statistics to support your findings and elaborate on any observed patterns or trends. **(/10)**
d. Ensure your presentation is professional and adhere to the standards required by scientific publications. State the major aims of your analysis and the patterns you seek. Using the combined findings from the EDA and the figures produced here, discuss the findings in a formal Results section. **(/5)**

5.3 Bar Graphs

Bar graphs need stricter handling than they usually get. Use them confidently for **counts** and **proportions** of categorical data. Use them more cautiously for **means with error bars**, because that display hides the shape of the underlying distribution. When the data are continuous and the distribution is important, box plots or violin plots are usually better.

The first use is counts or proportions in non-overlapping categories. Here the bar height is the quantity of interest. In the second use, a bar may represent a summary such as a mean with an error bar. That can be acceptable, but it should be a deliberate choice rather than the default.

In Figure 7 I demonstrate count and proportion bar graphs using the built-in `iris` data:

```
iris.cnt <- iris %>%
  count(Species) %>%
  mutate(prop = n / sum(n))
```

```
plt1 <- ggplot(data = iris.cnt, aes(x = "", y = n, fill = Species)) +
  geom_bar(width = 1, stat = "identity") +
  labs(title = "Stacked Bar Graph", subtitle = "Counts",
       x = NULL, y = "Count") +
  theme_pubclean() + scale_color_few() +
  scale_fill_few()

plt2 <- ggplot(data = iris.cnt, aes(x = "", y = prop, fill = Species)) +
  geom_bar(width = 1, stat = "identity") +
  scale_y_continuous(breaks = c(0.00, 0.33, 0.66, 1.00)) +
  labs(title = "Stacked Bar Graph", subtitle = "Proportions",
       x = NULL, y = "Proportion") +
  theme_pubclean() + scale_color_few() +
  scale_fill_few()

plt3 <- plt1 + coord_polar("y", start = 0) +
  labs(title = "Pie Chart", subtitle = "Same proportions, weaker comparison",
       x = NULL, y = NULL) +
  scale_fill_brewer(palette = "Blues")

plt4 <- ggplot(data = iris, aes(x = Species, fill = Species)) +
  geom_bar(show.legend = FALSE) +
  labs(title = "Side-by-Side Bars", subtitle = "Counts per species", y = "Count") +
  theme_pubclean() + scale_color_few() +
  scale_fill_few()

ggarrange(plt1, plt2, plt3, plt4, nrow = 2, ncol = 2,
          labels = "AUTO", common.legend = TRUE)
```

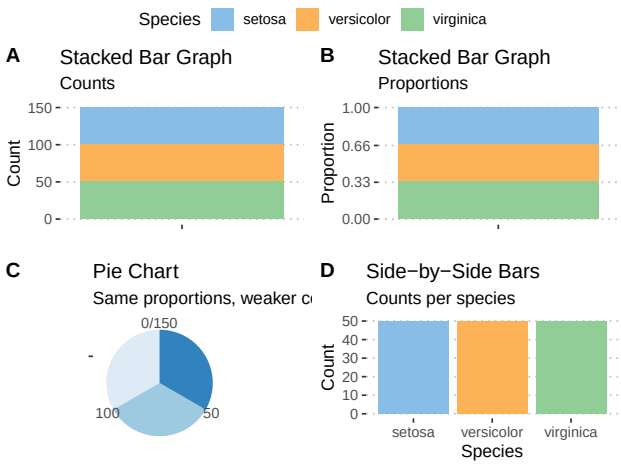

Figure 7. Bar graphs for categorical summaries in the built-in *Iris* data. A) A stacked count bar graph. B) A stacked proportion bar graph. C) A pie chart using the same proportions. D) A side-by-side count bar graph.

Here is the second use, where the bars represent means with standard deviations, as shown in Figure 8:

```
iris ▷
  pivot_longer(cols = Sepal.Length:Petal.Width,
               names_to = "variable",
               values_to = "size") ▷
  group_by(Species, variable) ▷
  summarise(mean = round(mean(size), 1),
            sd = round(sd(size), 1), .groups = "drop") ▷
  ggplot(aes(x = Species, y = mean)) +
  geom_bar(position = position_dodge(), stat = "identity",
           col = "black", fill = "salmon", alpha = 0.4) +
  geom_errorbar(aes(ymin = mean - sd, ymax = mean + sd),
                width = .2) +
  facet_wrap(~variable,
             scales = "free") +
  ylab("Size (mm)") +
  theme_grey()
```

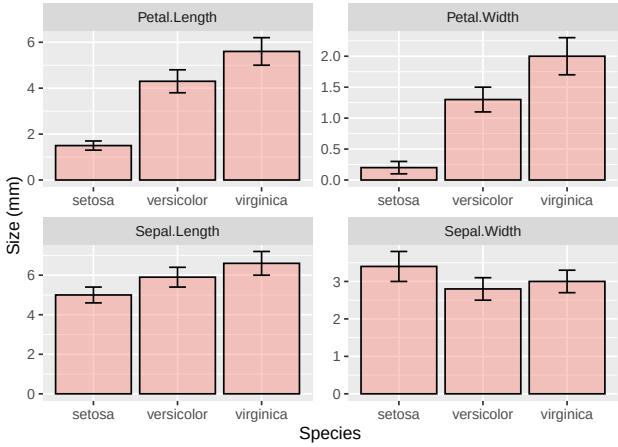

Figure 8. Bar graphs indicating the mean size (± SD) for various flower features of three species of *Iris*.

The rule to keep in mind is to use bar graphs for counts or proportions, and to use box plots or violin plots when you need to show how continuous data are distributed within groups.

DO IT NOW!

Open a recent biology paper (your own reading list, or ask me for one). Find a bar graph that shows mean ± SD (or ± SE) for a continuous variable across treatment groups. Ask yourself: (1) does the bar graph reveal the distribution shape or just a summary? (2) What plot type would be more informative? Can you redraw the same comparison as a box plot or violin plot using the approximate data you can read from the original figure (you'll need n, the mean, and the SD)? If you can, would your interpretation of the results change?

6 Visualising Relationships

We now examine relationships between variables. Here the choice depends on whether the observations are scattered in two dimensions or ordered along time or some other continuous sequence.

6.1 Scatter Plots

Scatter plots are the standard choice for two continuous variables. Each point is one observation, and the cloud of points shows association, spread, outliers, clustering, and possible nonlinearity. Because the points are not connected, the plot does not imply continuity between neighbouring observations.

Scatter plots are especially useful for:

- identifying positive, negative, or absent trends;
- assessing whether the relationship is roughly linear or clearly nonlinear;
- revealing clusters or subgroup structure;
- checking for outliers that could influence later analyses.

All of these are illustrated in Figure 9. The example shows the relationship between two continuous variables in the `iris` data. The grouping structure is shown with colour, and the second panel adds fitted lines.

```
plt1 <- ggplot(data = iris, aes(x = Petal.Length, y = Petal.Width, colour = Species)) +
  geom_point() +
  labs(x = "Petal length (cm)", y = "Petal width (cm)") +
  scale_color_fivethirtyeight() +
  scale_fill_fivethirtyeight()

plt2 <- ggplot(data = iris, aes(x = Petal.Length, y = Petal.Width, colour = Species)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, show.legend = FALSE) +
  scale_color_fivethirtyeight() +
  scale_fill_fivethirtyeight() +
  labs(x = "Petal length (cm)", y = "Petal width (cm)")

ggarrange(plt1, plt2, ncol = 2, nrow = 1, labels = "AUTO", legend = "right",
          common.legend = TRUE)
```

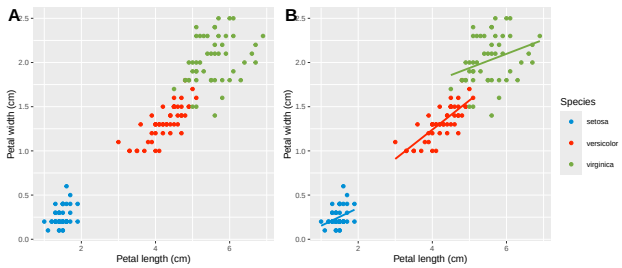

Figure 9. Examples of scatter plots made for the *Iris* data. A) A default scatter plot showing the relationship between petal length and width. B) The same data with fitted lines added.

These same features make scatter plots useful later as diagnostic tools for modelling.

DO IT NOW!

Using the built-in `msleep` dataset, make a scatter plot of `bodywt` (x-axis) against `brainwt` (y-axis). Colour the points by `vore`. Then add a separate linear trend line for each feeding group using `geom_smooth(method = "lm", se = FALSE)`.
Describe what you see: do the feeding groups appear to follow the same body-mass to brain-mass relationship, or do some groups differ? Are there any obvious outliers, and do the points cluster more strongly in some groups than others? Does adding grouped trend lines change your interpretation compared with looking at the scatter alone? You will revisit this kind of plot when we fit regression models in Chapter 12.

6.2 Line Graphs

A line graph connects data points with lines and is most useful when the order of the observations must be considered. The usual case is time, but any ordered continuous scale can serve the same role. A line graph therefore implies continuity from one observation to the next in a way that a scatter plot does not.

We typically encounter line graphs in time-series displays. Points may be shown or omitted. Importantly, the line connects consecutive observations and makes trends, cycles, and seasonality easier to see. The monthly climatology in Figure 10 provides a simple example.

```
library(lubridate)
read_csv(here::here("data", "BCB744", "SACTN_SAWS.csv")) ▷
  mutate(month = month(date)) ▷
  group_by(site, month) ▷
  dplyr::summarise(temp = mean(temp, na.rm = TRUE), .groups = "drop") ▷
  ggplot(aes(x = month, y = temp)) +
  geom_step(colour = "red4") +
  scale_x_continuous(breaks = c(1, 3, 5, 7, 9, 11)) +
  xlab("Month") + ylab("Temperature (°C)") +
  facet_wrap(~site, ncol = 5) +
  theme_grey()
```

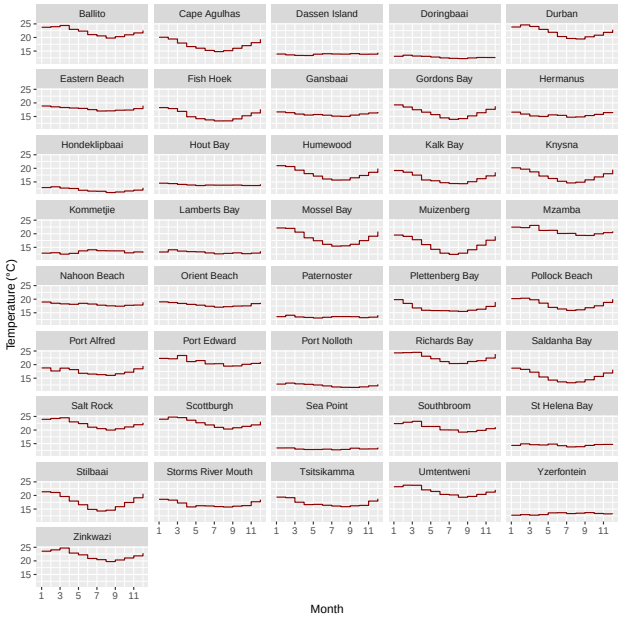

Figure 10. A time-series plot showing the monthly climatology for several sites around South Africa. The specific line drawn here forms a stepped graph.

7 Extensions

These plots extend the same principles to higher-dimensional data. They are still descriptive tools, but they are designed for cases where a simple one-dimensional distribution or two-variable relationship is no longer enough.

7.1 Heatmaps

Heatmaps encode values in a matrix as colour. They are useful when the structure of interest lies across two ordered dimensions at once, such as time by variable, site by species, or a correlation matrix. We will see another heatmap in Chapter 9 on Correlation and association. A calendar heatmap is a specialised form that lays time out in calendar space rather than on a single axis. The Western Australian example in Figure 11 shows how this reveals clustering in time. The Western Australian example in Figure 11 shows how this reveals clustering in time.

```
source("https://raw.githubusercontent.com/iascchen/VisHealth/master/R/calendarHeat.R")

temps <- heatwaveR::sst_WA ▷
  filter(t ≥ "2010-01-01" & t ≤ "2019-12-31") ▷
  mutate(weekday = lubridate::wday(t),
         weekday_f = lubridate::wday(t, label = TRUE),
         week = lubridate::week(t),
         month = lubridate::month(t, label = TRUE),
```

```
         year = lubridate::year(t)) ▷
  group_by(year, month) ▷
  mutate(monthweek = 1 + week - min(week))

ggplot(temps, aes(monthweek, weekday_f, fill = temp)) +
  geom_tile(colour = "white") +
  facet_grid(year(t) ~ month) +
  scale_x_continuous(breaks = c(1, 3, 5)) +
  scale_y_discrete(breaks = c("Sun", "Wed", "Sat")) +
  scale_fill_viridis_c() +
  xlab("Week of Month") +
  ylab("") +
  ggtitle("Time-Series Calendar Heatmap: Western Australia SST") +
  labs(fill = "[°C]")
```

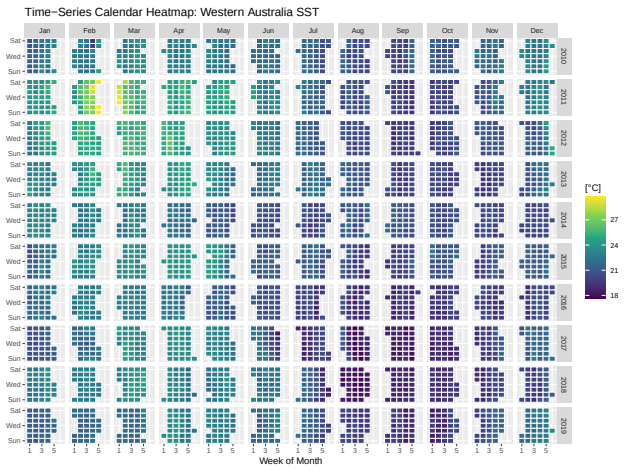

Figure 11. A calendar heatmap showing a time series of SST for Western Australia. The marine heatwave of summer 2011 is visible in the warm colours.

7.2 Hovmöller Diagrams

A Hovmöller diagram is a specialised heatmap used widely in oceanography and atmospheric science. One axis is usually time and the other is a spatial coordinate such as latitude or longitude. This makes it useful for tracking how spatial structure changes through time.

A **horizon plot** is a related way to compress long time-series displays while preserving anomaly structure. I provide more detail in the vignette, where the method is used for extreme temperature events.

A Hovmoller diagram such as Figure 12 makes the latitude-by-time structure explicit.

```
library(data.table)
library(colorspace)

NWA <- fread(here::here("data", "BCB744", "NWA_Hovmoller.csv"))

NWA ▷
  mutate(anom = zonal_sst - mean(zonal_sst)) ▷
  ggplot(aes(x = t, y = lat, fill = anom)) +
  geom_tile(colour = "transparent") +
  scale_fill_binned_diverging(palette = "Blue-Red 3", n_interp = 21) +
  xlab("") + ylab("Latitude [°N]") + labs(fill = "[°C]") +
  theme_grey()
```

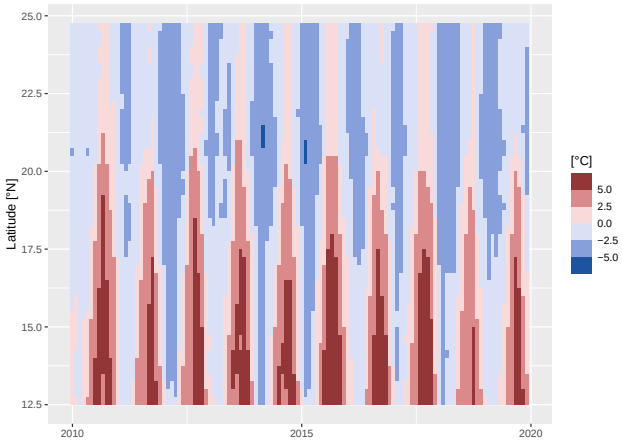

Figure 12. A Hovmöller diagram of zonally averaged SST for a region off Northwest Africa in the Canary upwelling system.

```
library(ggHoriPlot)

cutpoints <- NWA  %>%
  mutate(
    outlier = between(
      zonal_sst,
      quantile(zonal_sst, 0.25, na.rm = TRUE)-
        1.5*IQR(zonal_sst, na.rm = TRUE),
      quantile(zonal_sst, 0.75, na.rm = TRUE)+
        1.5*IQR(zonal_sst, na.rm=TRUE))) %>%
  filter(outlier)

ori <- round(sum(range(cutpoints$zonal_sst))/2, 2)

sca <- round(seq(range(cutpoints$zonal_sst)[1],
                 range(cutpoints$zonal_sst)[2],
                 length.out = 7)[-4], 2)

NWA %>% ggplot() +
  geom_horizon(aes(t,
                   zonal_sst,
                   fill = after_stat(Cutpoints)),
               origin = ori, horizonscale = sca) +
  scale_fill_hcl(palette = "RdBu", reverse = TRUE) +
  facet_grid(lat~.) +
  theme_few() +
  theme(
    panel.spacing.y = unit(0, "lines"),
    strip.text.y = element_text(size = 7, angle = 0, hjust = 0),
    axis.text.y = element_blank(),
    axis.title.y = element_blank(),
    axis.ticks.y = element_blank(),
    panel.border = element_blank()
    ) +
  scale_x_date(expand = c(0,0),
               date_breaks = "1 year",
               date_labels = "%Y") +
  xlab("Year") +
  ggtitle("Canary current system zonal SST")
```

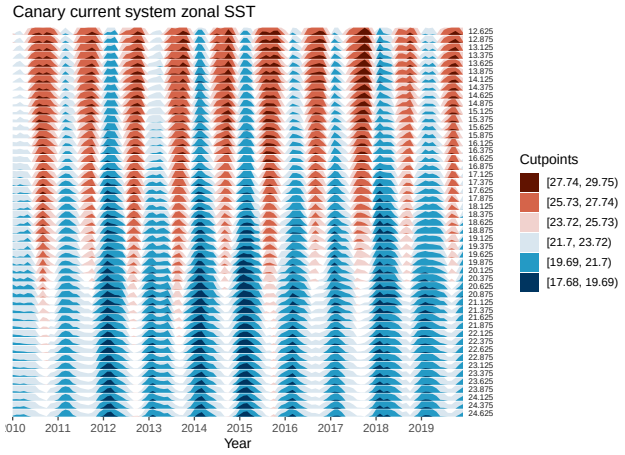

Figure 13. Zonally averaged time series of SST in the Canary Current system displayed as a horizon plot.

The same zonal SST structure can also be compressed into the horizon display in Figure 13.

7.3 Other Examples (To Develop)

(Incomplete.)

8 Conclusion

Figures extend statistical summaries by showing what the summaries hide. The main task is to match the plot to the question: distributions with histograms or density plots, group comparisons with box or violin plots, relationships with scatter plots or line graphs, and more complex structures with heatmaps or related displays.

The list in this chapter is not exhaustive, but it covers the main decisions you will make during EDA. For more examples and implementation detail, return to the resources listed in the introduction.